Week 4: When Constraints Break

Local Search and Hill Climbing

FIT1061: Introduction to Artificial Intelligence

2026-08-03

The Story: The Southwest Airlines Collapse

Between December 21 and 29, 2022, Southwest Airlines cancelled more than **16,700 flights** and stranded **over 2 million passengers**. A winter storm hit the United States just before Christmas, but every airline flew into the same storm. The others recovered in two or three days. Southwest took **ten**. At the peak, Southwest cancelled more flights than every other US carrier *combined*.

The storm was the trigger, not the cause. To see why Southwest broke so badly, you have to look at *how it schedules planes and crews*.

Most large airlines fly **hub-and-spoke**: flights route through a few big hubs. A storm in Denver delays Denver, but the rest of the network keeps running around it. Southwest flies **point-to-point**: planes hop from city to city in long chains. A plane delayed in Denver was supposed to fly Phoenix to Nashville next, then somewhere else after that. When that first leg breaks, the crew is now in the wrong city, and so is the next crew, and the next. The delays *cascade*.

Behind the scenes, software was supposed to fix this. When flights cancel, a scheduling system searches for a new, workable arrangement of planes and crews. But when 700-plus flights cancel in a single day, *every* nearby arrangement runs into a wall: a crew that has run out of legal duty hours, a plane in the wrong place, a gate that is already taken. The system had nowhere left to move. It was not that the algorithm was broken. It was that the algorithm was **stuck**.

That word (*stuck*) is what this week is about. We will build the simplest algorithm that can get stuck, watch exactly *how* it gets stuck, and see one clever trick that gets it unstuck. Then we will return to Southwest and ask the harder question: **when an algorithm fails at this scale, who is responsible?**

From Paths to Configurations

For the past two weeks, every search we ran asked the same question: *what is the path from A to B?* Breadth-first search, depth-first search, greedy best-first: all of them explored a **graph** by growing a **frontier**, the collection of places waiting to be explored. The answer they returned was a *route*: this step, then this step, then this step, until the goal.

But not every problem is a route. Consider these:

The problem	The answer is a…	Does the path matter?
Place 8 queens so none attack	**board**	no
Assign every exam to a timetable slot	**timetable**	no
Roster crews and planes onto a schedule	**schedule**	no

In each of these, we only care about the **final configuration**. Nobody asks *how* you arrived at a valid 8-queens board; they only ask whether it is valid. There is no meaningful "route" to report.

This changes everything about how we should search. A frontier holds *partial journeys* so we can extend them and remember how we got somewhere. But if the journey does not matter, the frontier is doing the **wrong job**: storing bookkeeping we will never use. So we drop it.

> **i** A family with a name: constraint satisfaction
>
> Many configuration problems share a shape called a **constraint satisfaction problem (CSP)**:
>
> - a set of **variables** (e.g. the 8 rows of a chessboard),
> - each with a **domain** of possible values (e.g. which column that row's queen sits in),
> - and a set of **constraints** saying which combinations are legal (e.g. no two queens attack each other).
>
> A solution is any assignment of values to variables that satisfies every constraint. The answer is a configuration, not a route.

For the rest of this week we work on the classic CSP: the **8-queens problem**. Place 8 queens on a chessboard so that no two share a row, column, or diagonal. (Posed in 1848; there are exactly 92 solutions on the 8×8 board.) It is small enough to hold in your head and hard enough to get stuck on.

Local Search and Hill Climbing

Local search keeps just *one* state in memory (one full board) and tries to improve it step by step. No frontier, no visited set. This is a big change from Weeks 2 and 3: because it never grows a frontier, its memory stays **flat** no matter how large the problem is. That is what lets it scale to problems where BFS would run out of memory instantly. Even for 8-queens, the space of one-queen-per-row boards is $8^8 \approx 17$ million states (vast, with no shortcut to the answer) which is exactly the regime where constant-memory local search earns its keep.

To turn "improve the board" into an algorithm, we need three ingredients.

State, score, and neighbours

State. One complete board. We use the convention that a board is a list of columns, one per row: `board[row] = column`. So `board = [3, 1, 6, 2, 5, 7, 4, 0]` means the queen in row 0 sits in column 3, the queen in row 1 sits in column 1, and so on. Because we put exactly one queen per row, we can never place two queens in the same row: one whole class of conflict is designed away for free.

Score (violations). We need a number that says how *bad* a board is. For 8-queens, the natural score is the number of **violations**: the count of *attacking pairs* of queens (two queens in the same column, or on the same diagonal). Two queens share a diagonal whenever the difference in their rows equals the difference in their columns: this covers the long diagonals too, not only queens on touching squares. A perfect solution has **0 violations**. Our goal is to drive the score down to zero.

Neighbours. From one board, which boards can we reach in a single small move? We say a neighbour is any board reached by **moving one queen within its own row** to a different column. Each of the 8 rows has 7 other columns it could move to, so every board has $8 \times 7 = 56$ neighbours. (The size of this neighbourhood matters: a bigger neighbourhood means more work to check at each step, but a smaller one makes it easier to get stuck with nowhere good to go.)

The loop

Hill climbing is now almost embarrassingly simple. Stand on your current board. Look at every neighbour. Move to the best one: the neighbour with the fewest violations. Repeat. Stop when **no neighbour is strictly better** than where you already are.

Definition: Hill Climbing

Repeatedly move from the current state to its **best neighbour**, as long as that neighbour is a *strict* improvement on the score. Stop when no neighbour improves: you are at a **local optimum**.
The name is a convention from problems where you climb *up* to a high score. Our problem runs the other way (we walk *downhill* in violations toward zero) but the algorithm is identical. High score good, or low score good: same loop.

In language-neutral pseudocode:

```
hill_climbing(board):
    current <- board
    loop:
        violations_here <- count_violations(current)
        neighbours      <- get_neighbours(current)
        best            <- the neighbour with the fewest violations
        if count_violations(best) >= violations_here:
            return current            # no strict improvement: local optimum
        current <- best
```

Three details are worth pinning down, because they are the most common places to go wrong:

- **Strict improvement.** We move only if the best neighbour is *strictly* better (`<`, not `<=`). If we allowed equally-good moves, the algorithm could wander forever between boards of the same score.
- **Ties.** If several neighbours tie for best, pick the first one you find. Any consistent rule works.
- **What it returns.** It returns a board, which may or may not be a valid solution. Sometimes the board it stops on still has violations. That is not a bug: it is the whole point of this week.

A Worked Trace: One Step on 4-Queens

The 8-queens board has 56 neighbours per step: too many to check by hand. So let us shrink to **4 queens** (a 4×4 board, $4 \times 3 = 12$ neighbours) and do *one* full step of hill climbing by hand.

Start from `board = [1, 2, 1, 3]`: queens at (row 0, col 1), (row 1, col 2), (row 2, col 1), (row 3, col 3):

```
. Q . .      row 0, col 1
. . Q .      row 1, col 2
. Q . .      row 2, col 1
. . . Q      row 3, col 3
```

Count the violations. We check every pair of queens for a shared column or diagonal:

- rows 0 and 1, columns 1 and 2, one apart in both row and column: **diagonal attack**.
- rows 0 and 2, both in column 1: **same-column attack**.
- rows 1 and 2, columns 2 and 1, one apart in both: **diagonal attack**.

That is **3 violations**. (The queen in row 3 is not attacking anyone.)

Look at the neighbours. A neighbour moves one queen along its row. There are 12 in total. A few of them:

Move	New board	Violations
row 0's queen: col 1 → col 0	`[0, 2, 1, 3]`	2
row 1's queen: col 2 → col 3	`[1, 3, 1, 3]`	2
row 2's queen: col 1 → col 0	**`[1, 2, 0, 3]`**	**1**
row 3's queen: col 3 → col 0	`[1, 2, 1, 0]`	5

Checking all 12, the best neighbour is `[1, 2, 0, 3]` with just **1 violation**. So hill climbing moves the row-2 queen from column 1 to column 0. The score dropped from 3 to 1 in one step:

```
. Q . .      row 0, col 1
. . Q .      row 1, col 2
Q . . .      row 2, col 0   <- moved
. . . Q      row 3, col 3
```

From here the algorithm would look again, find the best of *this* board's neighbours, and either improve once more or stop. That is the entire mechanic. **8-queens works exactly the same way**: there are just 56 neighbours to check per step instead of 12, which is why we let the computer do it.

Local Minima: Why It Gets Stuck

Hill climbing is fast and simple. It also has a fatal weakness, and it is worth seeing clearly because it comes back all semester.

Picture the score as a **landscape**. Every possible board is a point on the ground, and its height is its score. Because we are *minimising* violations, we want the lowest valley: a board of height zero. Hill climbing walks downhill: at each step it steps to the lowest neighbouring point it can see.

But hill climbing can only see its *immediate* neighbours. Imagine you are standing in a shallow dip. Every direction around you goes *up*. Hill climbing looks around, sees no lower neighbour, and **stops**, even though a much deeper valley sits just over the next ridge, out of view. That shallow dip is a **local minimum**: better than everything next to it, but not the best overall.

> **!** Key idea: local minima are geometry, not bugs
>
> When hill climbing gets stuck in a local minimum, the algorithm did **exactly what we asked**: always move to a strictly better neighbour, otherwise stop. The trap is not in the code. It is built into the *shape of the problem*. Some landscapes simply have valleys that are not the deepest valley, and a short-sighted downhill walker will fall into them.

For 8-queens, a local minimum looks like this: two queens are in conflict, but *every* single move you could make (shifting any one queen) creates a *different* conflict somewhere else. There is no improving move. You are stuck at, say, 1 or 2 violations, agonisingly close to a solution you cannot reach.

How often does this actually happen? We can just measure it. Run hill climbing from a large number of random starting boards and count how many reach zero violations.

Plain hill climbing **gets stuck about 86% of the time**: it solves the puzzle only about one run in seven. (This is the figure Russell & Norvig report from their textbook experiment; our own run of 200 random starts landed at about **84%** stuck, right next to theirs.) When it succeeds it takes about **4 steps**; when it gets stuck it gives up after about **3**. It fails fast, and often.

An 86% failure rate would be a disaster in any real system. But notice *why* it is bad: not because the algorithm is slow or complicated, but because it is **short-sighted**. So the fix should not make it smarter. It should just give it more chances.

Random Restarts

Here is the trick, and it is very simple.

If hill climbing gets stuck, **throw the board away and start again from a fresh random board.** Keep doing that until one of the runs reaches zero violations.

```
random_restart(k):
    for k attempts:
        result <- hill_climbing(random_board)
        if result is a solution:      # zero violations
            return result
    return the best board seen so far
```

Why does this work so well? Each run independently has roughly a 1-in-7 chance of landing in a good valley. The runs are independent, so the chance that *all* of them fail shrinks fast. One run succeeds about 14% of the time; after around **30 restarts** the chance of having found a solution is about **99%**. We did not make any single run smarter; we just kept rolling the dice.

> **!** Key takeaway
>
> Random restarts do not remove local minima; the landscape is unchanged. They *escape* local minima by trying many starting points and keeping the best. Local-search methods like this scale astonishingly well on the n-queens problem: versions with **millions of queens** can be solved in seconds.

Keep this trick in mind. The same "you got stuck? start over and try again" idea returns later in the semester, when we train models by walking downhill on a very different landscape.

When Constraints Break at Scale: Who Is Responsible?

Now go back to Southwest. The shape is the same as 8-queens. A schedule is a complete state. A crew-rest breach or an out-of-position plane is a violation. Rerouting one plane or swapping one crew is a move to a neighbour. And in late December 2022, the system reached a state where **every nearby schedule violated some constraint**: crew hours, aircraft position, gate availability. Every direction was uphill. The system had nowhere to move.

(Real-world failures are always more complicated than any one model; this is one lens, not the whole story. Southwest's actual collapse also involved two scheduling tools that did not coordinate with each other. But the intuition holds: *optimisation systems have trapping geometries, and at scale, getting stuck is expensive.*)

At 8-queens scale, getting stuck costs nothing: you restart and try again. At airline scale, the same geometry cost **roughly $725–800 million**, ten days of chaos, and eventually a **$140**

million US Department of Transportation penalty: the largest consumer-protection fine in US aviation history.

So here is the question this week leaves you with:

> **i** Who is responsible when an algorithm fails?
>
> - The engineers who built the scheduling system in the 1990s?
> - The executives who deferred a documented modernisation budget?
> - The regulators who never required resilience stress-testing?
> - The pilots' union, which had publicly warned for years that exactly this would happen?
> - Or is it a system failure with no single owner, responsibility spread thin across all of them?

There is no clean answer, and you are not expected to find one. But notice what made the question *possible*: the algorithm worked exactly as designed. The failure was upstream of the code: in decisions about money, staffing, and warnings ignored. If you only ever learn the algorithm, you cannot diagnose this kind of failure. That is why an AI course spends a page on an airline.

Summary

Key vocabulary:

Term	Meaning
Configuration problem	We want a final *arrangement*, not a route to it
Constraint satisfaction problem (CSP)	Variables with domains, plus constraints saying which combinations are legal
Local search	Keeps one state and improves it; no frontier, flat memory
State	One complete candidate (e.g. a full 8-queens board)
Violations (score)	How bad a state is; here, the number of attacking pairs
Neighbour	A state reached by one small change (move one queen in its row)
Hill climbing	Repeatedly move to the best strictly-better neighbour; stop otherwise
Local minimum	A state better than all its neighbours but not the best overall
Random restarts	Escape local minima by re-running from fresh random starts

Key takeaways:

- Some problems ask for a **configuration**, not a **path**. When the journey does not matter, drop the frontier.
- **Local search** keeps a single state and improves it, using constant memory regardless of problem size.
- **Hill climbing** needs three things: a *state*, a *score*, and a *neighbour* relation. Then it just walks toward the best neighbour until none improves.
- Hill climbing gets stuck in **local minima**, and this is *geometry, not a bug*. On 8-queens it fails about **86%** of the time.
- **Random restarts** escape local minima cheaply: many tries, keep the best. About 30 restarts give a ~99% success rate on 8-queens.
- At scale, the same "stuck" geometry drove the **Southwest 2022 collapse**, which raises the question of *who is responsible when an algorithm fails*.

What's next: we bring together all the search algorithms from Weeks 2 to 4 and turn to cases where the algorithm's choice has real consequences for real people.

In the Lab

This week's task: Hill Climbing on 8-Queens

1. **Hand-trace** one hill-climbing step on your personalised starting board: count the violations on your board and on a handful of candidate one-queen moves by hand, and identify the move hill climbing would make. Before any code.
2. **Explore** the provided helpers: `count_violations` (score a board) and `get_neighbours` (list the 56 neighbours). Make sure you understand what each returns.
3. **Implement** hill climbing by writing the loop from the pseudocode: score the current board, find the best neighbour, move if it is strictly better, otherwise stop.
4. **Run 50 trials** from random starting boards. Record how many solved (0 violations) and how many got stuck. Plot the violations-per-step for one solved run and one stuck run.
5. **Reflect** in 150–250 words: *Who is responsible when an algorithm fails?* Engage with the Southwest story, weigh at least two of the parties above, and commit to a position.

Aiming higher: any of these push past the plain Pass build.

- **Random restarts.** When a run gets stuck, throw the board away and start again from a fresh random board. Plot the success rate as the number of restarts grows (1, 5, 10, 20, 50, 100). How many restarts does it take to get above 95%?
- **Sideways moves.** Allow the algorithm to accept a neighbour with an *equal* score (not just a strictly better one), with a cap on how many consecutive sideways moves it may take before giving up. Does letting it shuffle across flat spots help it escape? On 8-queens

this single change lifts the solved rate dramatically (Russell & Norvig report a jump from 14% to 94%).

- **Graph-structure comparison.** Revisit breadth-first, depth-first, and greedy best-first from the past weeks and compare how each behaves as the *shape* of the search problem changes. Which algorithm wins on which structure, and why?